SLiM Workshop Series #21: Multispecies Modeling

Copyright © 2025 Benjamin C. Haller. All rights reserved.

1. Start with a new "no-genetics" model

We're going to build a multispecies model of foxes, mice, and predation, completely from scratch. In SLiMgui, open a new window, delete the default model, and type this instead:

```
species mouse initialize() {
}

ticks all 1 early() {
   mouse.addSubpop("p1", 5000);
}

ticks all 500 late() {
}
```

It may surprise you to learn that this is a complete model that runs in SLiM, since it has no initialization of genetics at all – no `initializeMutationType()`, `initializeGenomicElement()`, and so forth. This is called a "no-genetics model". No-genetics models aren't very useful when you want to model evolution, so they're not commonly used in SLiM; but they're useful if you want to model ecology, and we're going to start with ecology now.

Run the model now in SLiMgui to check it. Of course, it doesn't do much!

2. Add a second species

This model is also a multispecies model, technically; that's because it uses the `species mouse` and `ticks all` syntax of multispecies models, which flips SLiM over into multispecies mode. However, it has only one species, `mouse`.

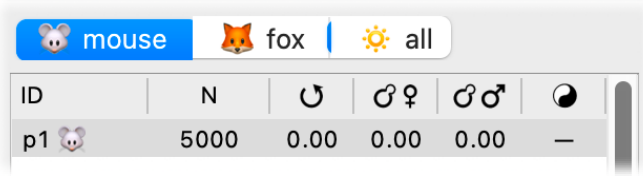
Let's fix that by fleshing out the model a bit. For one thing, we'll add a `species fox` and a subpopulation `p2` for our foxes. Also, let's add a call to `initializeSpecies()` for each species, and set an avatar and a color for each species. The avatar and color are used in SLiMgui for display purposes, as we'll see. If you're not sure how to type emojis on your computer, take a minute to find out (maybe google it – it depends on your operating system); the avatar emojis are not strictly necessary, but they do make the SLiMgui experience rather nicer. Here's the next version:

```
species mouse initialize() {
   initializeSpecies(avatar="🐭", color="black");
}
species fox initialize() {
   initializeSpecies(avatar="🦊", color="orange");
}

ticks all 1 early() {
   mouse.addSubpop("p1", 5000);
   fox.addSubpop("p2", 500);
}

ticks all 500 late() {
}
```

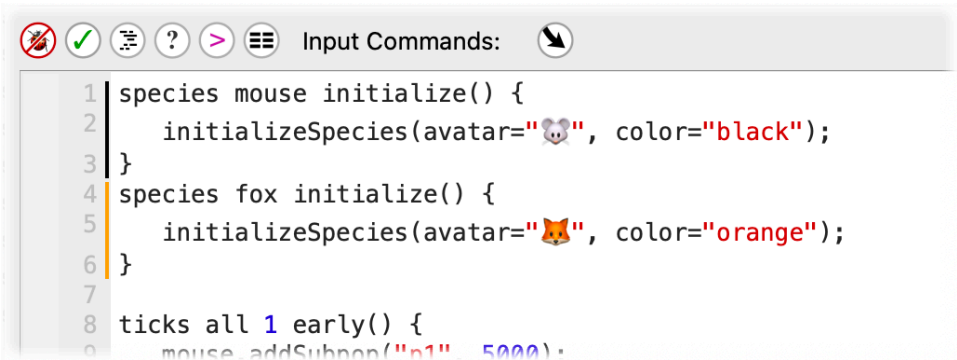
Recycle and run this model. You can see that SLiMgui notices that it has more than one species, and switches some of its user interface to facilitate our multispecies modeling. For one thing, the population view now has tabs, allowing us to select which species to view, or to view all of them:



Notice that SLiMgui is already using the avatars we chose, labeling the tabs and also the subpopulations with them. This makes it easier to tell what's going on, since many humans are quite visual – especially if you choose avatars that are different colors. Note that avatars don't need to be animal emojis; you can use any emoji, and you can use text instead, too. (If we hadn't set them, SLiMgui would've defaulted to A and B as the avatars for the two species.)

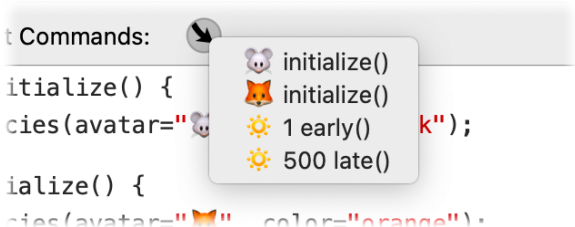
Click on the "all" tab to see how things look with that choice; notice you then get two Individuals views and two Chromosome views, one each for the mice and foxes.

Another way SLiMgui displays multispecies information is in the script editor:



The black and orange bars on the left show us that the two `initialize()` callbacks are species-specific code. They use the species colors that we set in our `initializeSpecies()` calls. Code that is not species-specific, such as the `ticks all 1 early()` event, does not get a color bar.

Finally, SLiMgui's Jump menu also gets a multispecies treatment:



OK; we have a mouse–fox multispecies model! We're going to want predation, so we need this to be a non-Wright–Fisher model so that the population size of the mice depends on how many get eaten, and the population size of the foxes depends on how many mice they manage to eat. Multispecies models are usually nonWF, for this sort of reason. So now let's convert the model to be nonWF.

EXERCISE: You might take a stab at doing the nonWF conversion yourself; it's good practice. We'll look at it in the next section, so if you're not sure how to proceed, just move on.

Hints: Make a `species all initialize()` callback where you define constants for things like the mouse and fox carrying capacities and mean fecundities (it can be nice to define all model parameters in one place). Add `ticks mouse early()` and `ticks fox early()` events for density-dependence. Add `species mouse reproduction()` and `species fox reproduction()` callbacks that just hermaphroditically generate a litter of offspring with a random mate.

3. Make it a nonWF multispecies model

The details of building a nonWF model should be fairly familiar by now, so we'll just look at the completed result (which you can get from the solutions if you don't wish to type it) and discuss a few points below. Here it is:

```
species all initialize() {
   defineConstant("K_mouse", 5000);
   defineConstant("K_fox", 500);
   defineConstant("F_mouse", 10);
   defineConstant("F_fox", 2);

   initializeSLiMModelType("nonWF");
}
species mouse initialize() {
   initializeSpecies(avatar="🐭", color="black");
}
species fox initialize() {
   initializeSpecies(avatar="🦊", color="orange");
}

species mouse reproduction()
{
   mate = subpop.sampleIndividuals(1, exclude=individual);
   if (size(mate))
      subpop.addCrossed(individual, mate, count=rpois(1, F_mouse));
}
species fox reproduction()
{
   mate = subpop.sampleIndividuals(1, exclude=individual);
   if (size(mate)) {
      subpop.addCrossed(individual, mate, count=rpois(1, F_fox));
}

ticks all 1 early() {
   mouse.addSubpop("p1", K_mouse);
   fox.addSubpop("p2", K_fox);
}

ticks mouse early() {
   p1.fitnessScaling = K_mouse / p1.individualCount;
}
ticks fox early() {
   p2.fitnessScaling = K_fox / p2.individualCount;
}

ticks all 500 late() {
}
```

We set up parameters and call `initializeSLiMModelType("nonWF")` in a `species all initialize()` callback. In the `reproduction()` callbacks, the focal reproducing mice/foxes choose a mate using `sampleIndividuals()`, excluding themselves to avoid selfing. They draw a litter size from a Poisson distribution using `rpois()`, and pass that to `addCrossed()` in the `count` parameter to generate that many offspring with their mate. Density dependence keeps the species populations close to their respective carrying capacities.

Note that the density dependence is done in `ticks mouse` and `ticks fox` events, separately. This is mostly just for illustration purposes; we could do the density dependence for both species in a single

`ticks all` event instead. Since the two species are on the same timescale, in this model, it doesn't matter. If we did decide to shift to using separate timescales, though, this design would be helpful.

Of course all sorts of biological details could be added to this, starting with separate sexes and going on to mate-choice behavior, monogamy or lack thereof, age structure, and so forth, but that's not our focus here. Instead, we want to get the ecology working: we want the foxes to eat the mice. The nonWF model above will be our foundation, going forward.

4. Add predation

So, we have a few design goals to think about. We want the mice to continue to have a top-down carrying capacity; their population size is limited by their (unmodeled) food source. We want the foxes, on the other hand, to have an *emergent* carrying capacity that is determined by how many of them survive and reproduce, which should depend upon how many mice they manage to eat. We want the foxes to count how many mice they eat; their survival and fecundity will then both depend upon that, making it doubly important that they succeed in hunting.

The necessary changes are not terribly large. First of all, add another constant, `HUNTS`; in each tick, the foxes will hunt this many times, so this is their maximum number of kills per tick:

```
defineConstant("HUNTS", 10);
```

Next, implement the hunting event:

```
ticks fox late() {
   // hunting: use tag values to track what happens
   // for mice: 0 means alive, 1 means killed
   // for foxes: the tag value is the number of mice killed
   c(p1, p2).individuals.tag = 0;

   // the foxes hunt in randomized order
   foxes = p2.sampleIndividuals(p2.individualCount);
   p_kill = min(1.0, p1.individualCount / K_mouse);

   for (hunter in foxes)
   {
      kill_count = rbinom(1, HUNTS, p_kill);
      killed = p1.sampleIndividuals(kill_count, replace=T);
      killed = unique(killed, preserveOrder=F);
      killed = killed[killed.tag == 0];
      killed.tag = 1;
      hunter.tag = size(killed);
   }

   all_killed = p1.subsetIndividuals(tag=1);
   mouse.killIndividuals(all_killed);

   p_fox_survival = foxes.tag / HUNTS;
   starved = runif(foxes.size()) > p_fox_survival;
   fox.killIndividuals(foxes[starved]);
}
```

We do hunting in a `late()` event so that it happens at the end of the tick, just before SLiMgui displays. That way we get to see the `mouse` and `fox` population sizes in SLiMgui post-hunting, which is more interesting and informative than seeing them after reproduction and then mortality (at which point the mice will typically be back at their carrying capacity and the effect of hunting on them will be hidden). The timing of various events could be rearranged, of course, to represent the biology of the system, and that rearrangement might have consequences for the behavior of the model; the order in which things happen is often quite important. Here, hunting occurs when the mice are

probably at carrying capacity (given their high fecundity). If the hunting logic were moved to an `early()` event the mice would be post-reproduction but pre-mortality, so they would be way above carrying capacity; the foxes would thus have more success with their hunts, and the mice would be affected less severely.

This block of code begins by zeroing out the `tag` values for both species, since those values are used to track the state of things internally as we will see. Next, the order of the foxes is shuffled by doing a full sample without replacement using `sampleIndividuals()`; this is just a precaution, to make sure that some kind of ordering bias doesn't distort the behavior of the model.

Then (skipping a line that will be explained shortly) we loop over the foxes, giving each one a chance to hunt. (Note that we can't call the loop index variable `fox`, because `fox` is already the name of the species object for foxes, so we use `hunter` instead.) Each focal fox calculates its number of kills using a binomial draw, where the number of trials is `HUNTS` and the probability of success per trial is `p_kill`. This explains the line we skipped before: `p_kill`, the probability of a fox getting a kill in a given pass, is the fraction of their carrying capacity that the mice are at. The main goal is that fox hunting success depends, in some way, on the density of the mice; making it the fraction of their carrying capacity, in particular, is an arbitrary choice that could be changed to match the biology.

Next, we figure out which mice were killed. We draw *with* replacement, for performance reasons (drawing without replacement is quite computationally expensive, for a large source vector); since that means we might get a duplicate, we then use `unique()` to unique down, but that shouldn't happen very often, so it's a reasonable approximation (note that `preserveOrder=F` is a good idea, for performance reasons, whenever you don't need `unique()` to preserve the order of the vector being uniqued). We then subset to select only mice that have not already been killed by a previous hunter; foxes hunting later in this `for` loop are at a disadvantage, with this particular hunting algorithm, which is one reason we shuffled their order as a precaution. One could design an order-free hunting algorithm, but it would probably be harder to explain in a worksheet.

In the last part of the loop, we update `tag` values: mice that were killed get set to `1`, and foxes get set to the number of mice they killed.

After the loop, we use those `tag` values to implement mortality. First we get all of the mice with a `tag` of `1`, and kill them with `killIndividuals()`. Then we calculate the survival probability for each fox from the fraction of its hunts that were successful, using a vectorized call to `runif()` to decide which foxes therefore starved, and call `killIndividuals()` to kill those foxes. We haven't seen `killIndividuals()` much; it was added in SLiM 4 specifically because it is so common, in multispecies models, to want to kill individuals directly, due to interspecific interactions, rather than waiting for the mortality stage of the tick cycle. The individuals passed are killed immediately and removed from their subpopulation(s). It might be possible to redesign the model to use `fitnessScaling` or a `survival()` callback instead, but it would be very inconvenient, since the survival/mortality tick cycle stage happens at a different time. Several different paths to mortality are provided by SLiM because mortality happens in different ways and at different times; you should choose the model implementation that is convenient and aesthetic.

OK, that's it for the hunting event. Since it took care of killing foxes that didn't hunt well enough, you should now **remove the fox density-dependence event**; they no longer have an intrinsic carrying capacity. The constant `K_fox` is now used only for the foxes' initial population size.

The only other change is a tweak to the litter size calculation in the fox `reproduction()` callback:

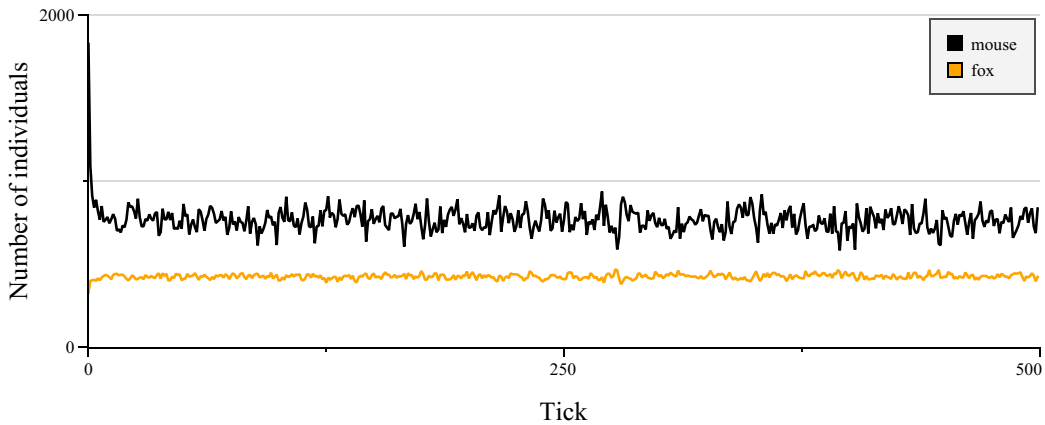
```
fecundity = F_fox * (individual.tag / HUNTS);
litterSize = rpois(1, fecundity);
```

This scales the focal fox's base fecundity `F_fox` by multiplying it by its fractional hunting success, `(individual.tag / HUNTS)`. This penalizes the foxes a second time for hunting poorly; even if a malnourished fox survives, its litter size (at least the number that survive to adulthood) will be

smaller. Again, this model just gestures towards the biology of the system; much more biological realism could be added if needed. (But always remember that more realism is not always better; in general, you should write the simplest possible model that serves to answer your research question, whatever that might be!)

That's everything we need for predation. The hunting code is not completely trivial, but once you understand the logic it is quite straightforward. Implementing other interactions, such as resource competition, parasitism, or mutualism, is often similarly straightforward.

Let's see how it behaves. In SLiMgui, click the graph button and choose "Multispecies Population Size ~ Time" to open that graph window. Then recycle and run the model, and you should see something like this (you can resize the graph window as desired, of course):



It seems to be quite stable; the mice and foxes both exhibit a fairly constant population size after the model equilibrates, with only a little stochastic variation.

EXERCISE: Why do you think the model is stable? What prevents extinction from occurring? Open a new window, copy-paste your model script over, and try playing with the model parameters a little. Can you get it to go extinct? Can you get the population sizes of the two species to cycle without going extinct? (There's a lot of adjustments that could be made; the solution will show one way of getting large-magnitude cycles.) Once you're done, close the copy to revert your changes.

5. Add genetics

So far we've just been modeling ecology, and this is often a good modeling strategy – get the ecology working first, and then add the genetics you're interested in. This avoids excessive complexity in your model code while you are still polishing the basic biology of reproduction, life history, population regulation, dispersal, and so forth.

So, having gotten the ecology working nicely, let's add genetics. Of course we could get into a lot of complexity here, with mouse and fox genetic maps and so forth, but here we are interested in modeling the coevolution between them, not fox and mouse genetics in general, so we'll keep things simple. Let's give the mice a Mendelian coat-color trait governed by a locus C, with incomplete dominance: they can be white (cc), light brown (Cc), or dark brown (CC). The foxes will get a quantitative trait representing the mouse coat color that a given fox looks for when hunting; they will tend to kill mice that match their trait value. (We'll call cc `0.0`, Cc `0.5`, and CC `1.0`).

There are several steps to be taken in implementing this. Let's start by adding the Mendelian trait for the mice. Define the genetics for `mouse`, in its `initialize()` callback:

```
species mouse initialize() {
   initializeSpecies(avatar="🐭", color="black");
```

```
   initializeMutationType("m1", 0.5, "f", 0.0);      // coat-color locus
   initializeGenomicElementType("g1", m1, 1.0);
   initializeGenomicElement(g1, 0, 0);
   initializeMutationRate(0.0);
   initializeRecombinationRate(0.0);
}
```

After the call to `addSubpop()` for the mice, set up their initial genetics by making the C allele be at 50% frequency in the population (roughly at Hardy–Weinberg equilibrium):

```
   targets = sample(p1.haplosomes, K_mouse);
   targets.addNewDrawnMutation(m1, 0);
```

Now define genetics for `fox`, in its `initialize()` callback:

```
species fox initialize() {
   initializeSpecies(avatar="🦊", color="orange");
   initializeMutationType("m2", 0.0, "n", 0.0, 0.1); // hunting
   m2.convertToSubstitution = F;
   m2.mutationStackPolicy = "l";
   initializeGenomicElementType("g2", m2, 1.0);
   initializeGenomicElement(g2, 0, 99);           // 100 QTLs
   initializeMutationRate(1e-5);                  // 1e-3 per gamete
   initializeRecombinationRate(0.5);              // unlinked
}
```

After the call to `addSubpop()` for the foxes, set up their genetics by giving them some initial genetic diversity. We will give every fox haplosome a QTL mutation with effect size `0.25`, making them all seek Cc mice initially. Above and beyond that, we will sprinkle 50 other QTLs at random positions in the haplosome, with random effect sizes, at random frequencies:

```
   p2.haplosomes.addNewMutation(m2, 0.25, 0);
   for (i in 1:50)
   {
      target = sample(p2.haplosomes, rdunif(1, 0, K_fox*2));
      target.addNewDrawnMutation(m2, rdunif(1, 1, 99));
   }
```

Because this is a quantitative trait and we're keeping the QTL effect sizes in the `selectionCoeff` property, we need to tell SLiM that they are neutral by adding a `mutationEffect()` callback:

```
species fox mutationEffect(m2) {
   return 1.0;             // QTLs are neutral
}
```

Finally, make the fox hunting depend upon the phenotypic match between the foxes and the mice. Add a chunk of code at the top of the hunting event that calculates phenotypes from genetics, and stores the phenotypes in the `tagF` properties of the individuals:

```
   // use tagF to hold individual phenotypes
   mice = p1.individuals;
   pheno = mice.countOfMutationsOfType(m1) / 2;
   mice.tagF = pheno;

   foxes = p2.individuals;
   pheno = foxes.sumOfMutationsOfType(m2);
   foxes.tagF = pheno;
```

And modify the hunting loop to use those `tagF` phenotypes:

```
for (hunter in foxes)
{
   kill_count = rbinom(1, HUNTS, p_kill);
   killed = p1.sampleIndividuals(kill_count, replace=T);
   killed = unique(killed, preserveOrder=F);
   killed = killed[killed.tag == 0];
   match = 1.0 - abs(hunter.tagF - killed.tagF) * MATCH_SCALE;
   match = pmax(pmin(match, 1.0), 0.0);
   killed = killed[runif(size(killed)) < match];
   killed.tag = 1;
   hunter.tag = size(killed);
}
```

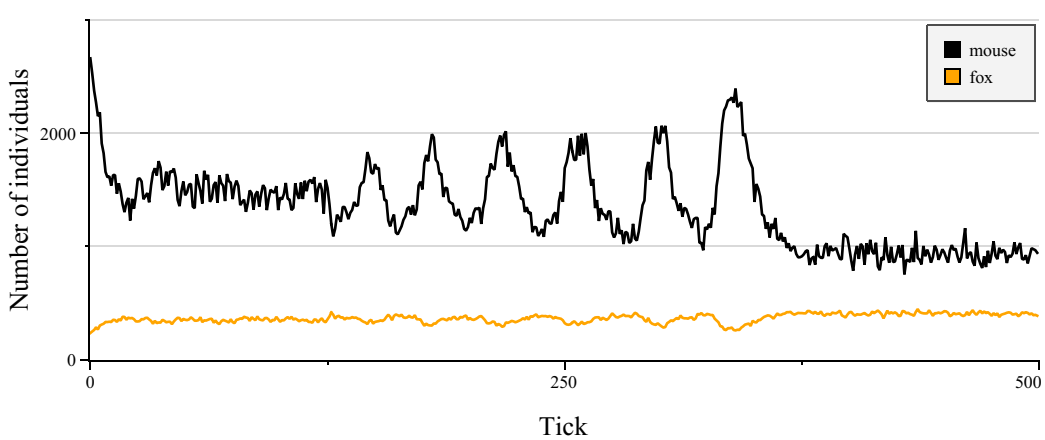
The `MATCH_SCALE` constant should be added to the definitions at the top:

```
defineConstant("MATCH_SCALE", 0.5);
```

This hunting loop code involves a couple of new steps. We sample, unique, and remove mice that have already been killed, as before. We now have a pool of mice that the focal fox might see and kill; next, we want to assess the visual match between the mouse coat colors and the fox's (genetically determined) search image. We calculate that match, in vectorized fashion, as `1.0` minus the absolute difference between the two quantities (scaled by `MATCH_SCALE`, which scales the importance of trait-matching in the foxes' hunting – the strength of selection that the mice and foxes exert upon each other, in a sense). That match is clamped to `[0, 1]` with `pmin()` and `pmax()` to provide a probability; we then keep each element of `killed` with a probability equal to the closeness of the match. In summary, the better the visual match between a particular mouse and the focal foxes' search image, the higher the probability that that mouse will be killed. This yields the final vector of mice killed by the focal fox; we then proceed as before.

Try recycling and running the model several times. It has a couple of behaviors, depending in part upon what QTLs happen to arise for the foxes. Things happen very quickly; you might use the play speed slider to slow the model down, or even single-step to watch what happens. It is helpful to have the Multispecies Population Size ~ Time plot open so you can see what is happening there. The mice often become monomorphic (although not always); what drives that to happen? This is an interesting model because the evolutionary dynamics destabilize the ecological dynamics, making the population sizes fluctuate wildly sometimes. Eco-evolutionary dynamics in action!

Here is one example time course, in which the mice become monomorphic toward the end:



These population size swings are generally coupled to wild swings in the phenotypic distribution as well. We could make a custom plot of that fairly easily – we already calculate mean phenotypes in each tick – but this worksheet has gone long enough, and we already saw custom plotting in

SLiMgui in the **SLiMgui Graphing** worksheet, so we will leave that as an exercise for the reader. (One very easy way to do it would be to output the mean phenotypes to a `LogFile` instance; SLiMgui could then make a plot for us directly from the logged data, as we saw previously in the "`modifyChild()` callbacks" worksheet.)

EXERCISE: The behavior of the model is sensitive to `MATCH_SCALE`. Try changing it to `0.0`; what do you observe? That value turns off trait-based matching in the hunting completely, essentially recovering the behavior of the no-genetics model (there are still genetics, but they are now neutral, and just drift). Try changing it to `1.0` now, and do several runs. How has the behavior of the model changed, compared to when the value was `0.5`? What do you think is happening?

EXERCISE: It would be nice to have colors for the individuals, to visualize which mice are cc/Cc/CC, and what quantitative trait values the foxes have. An easy way to implement this uses the Eidos function `colors()`; you can pass the phenotypes right in, since they are already on a [`0`, `1`] scale, and choose a color palette you like, such as `"plasma"`. The solution implements a fancier color scheme.

6. Add spatiality

As the final step with this multispecies model, let's add spatiality as well. The changes required are not difficult, but they're fairly extensive and it's basically review from the continuous-space section of the workshop, so we won't do them step by step here; instead, please open the solution for this section and look at it. In summary, here's what was done:

(1) Declare each species to be spatial (with periodicity) using `initializeSLiMOptions()`.

(2) Set random initial positions for all individuals when the subpopulations are created, using `setSpatialPosition()` and `pointUniform()`.

(3) Implement dispersal for the offspring individuals (for both species) using `subsetIndividuals()` and `deviatePositions()`.

(4) Set up mating interaction types for the two species (note that `InteractionType` is considered non-species-specific, because an interaction type can involve more than one species, so you set those up in your `species all initialize()` callback).

(5) Use those interaction types to choose mates spatially, with `drawByStrength()`.

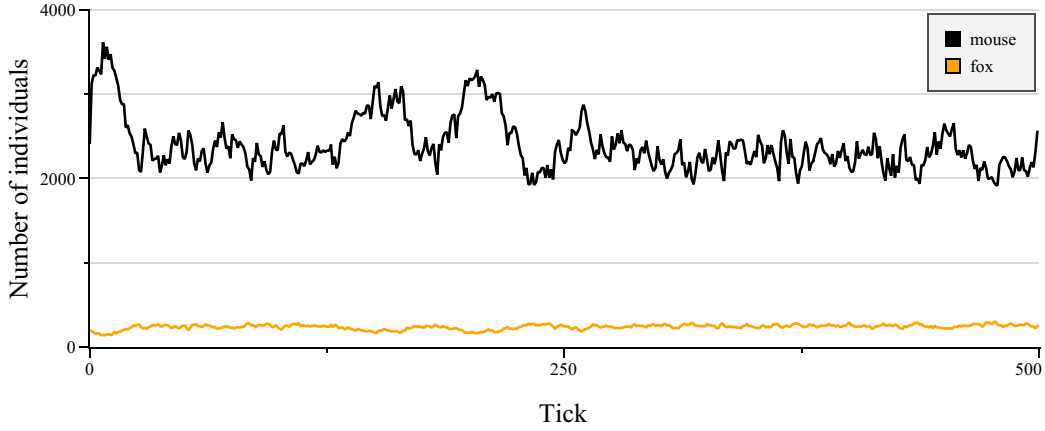
(6) Set up an interaction type for local competition between the mice as well, to implement spatial density-dependence.

(7) Implement that local competition using `localPopulationDensity()` and `fitnessScaling` – similar to the existing density-dependence code but with spatiality.

(8) Set up an interaction type for local hunting by the foxes.

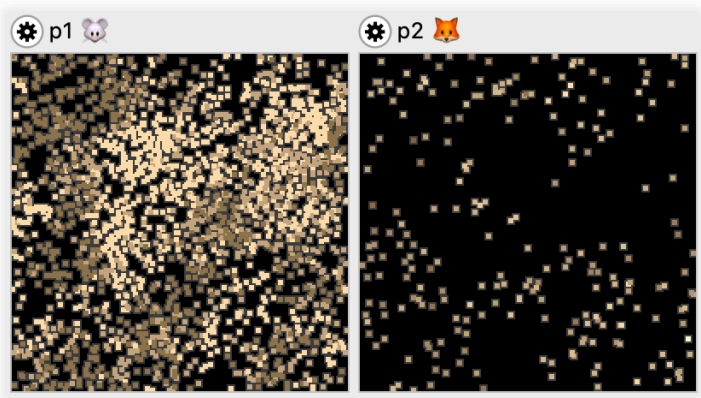
(9) Replace the `sampleIndividuals()` call in the hunting code with a `drawByStrength()` call. This interaction will involve both species, so you will evaluate it with `i4.evaluate(c(p1, p2))`, and you will pass `p2` to `drawByStrength()` as the `exerterSubpop`. Note that `drawByStrength()` never returns duplicates, so you can get rid of the `unique()` call.

Having opened the solution in SLiMgui, switch to the "all" tab so you can see Individual views for both species, and open up the Multispecies Population Size ~ Time plot. Give it a few runs. Here's an example run:



This seems qualitatively different from previous versions of the model, to me at least. Is the model unstable or stable? More specifically, does either species go extinct? Is the genetic diversity retained or lost? Why do you think the behavior is so different from the previous models?

The Individual views are quite informative, with the nice color scheme:



Strong spatial heterogeneity in the coat color of the mice can be seen. The foxes have evolved strongly towards hunting darker mice, so in areas where the mice are predominantly light, there are few foxes present. Of course this situation is temporary! Effects of spatial heterogeneity like this make spatiality extremely important in evolutionary ecology.

Please do your **BEEP!** now; this worksheet is already quite long. After that, you might want to try playing around with some of the parameters to see different ways that the model can behave. It's a pretty simple model, in the sense that it is only 2–3 pages of code, but it implements multispecies spatial eco-evolutionary dynamics, which is not a simple thing!

7. BEEP! With extra time, maybe look at multispecies models in chapter 20 of the manual. One interesting thing to look at is that section 20.3 shows how to implement a deterministic analytical multispecies model in SLiM, and then section 20.4 transforms that into an individual-based model. That is a common modeling problem, and we didn't look at it in this model – we started off already in the individual-based paradigm. Very often you have an existing analytical model that you want to turn into an individual-based model so that you can extend it, such as by making it spatial.

Alternatively, advanced users might try making a custom plot in SLiMgui using `LogFile` output. Log the mean phenotypes every tick with `LogFile`. Instead of making a plot with the context menu in the debugging output window, though, try doing it in script, using `Plot`. There is a method on the `SLiMgui` class, `logFileData`, that will provide the data. Try to put the pieces of the puzzle together.